

In [1]:

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import seaborn as sns; sns.set()
from sklearn.datasets import load_digits
```

In [2]:

```
breast = load_breast_cancer()
breast_data = breast.data
```

In [3]:

```
breast_data.shape
```

Out[3]:

```
(569, 30)
```

In [4]:

```
breast_labels = breast.target
```

In [5]:

```
breast_labels.shape
```

Out[5]:

```
(569,)
```

In [6]:

```
labels = np.reshape(breast_labels,(569,1))
```

In [7]:

```
all_breast_data = np.concatenate([breast_data,labels], axis=1)
```

In [8]:

```
#cheeck to see the columns merged
all_breast_data.shape
```

Out[8]:

```
(569, 31)
```

In [9]:

```
#create a new dataframe with all breast cancer data
breast_df = pd.DataFrame(all_breast_data)
```

In [10]:

```
features = breast.feature_names
```

In [11]:

```
features
```

Out[11]:

```
array(['mean radius', 'mean texture', 'mean perimeter', 'mean area',
      'mean smoothness', 'mean compactness', 'mean concavity',
      'mean concave points', 'mean symmetry', 'mean fractal dimension',
      'radius error', 'texture error', 'perimeter error', 'area error',
      'smoothness error', 'compactness error', 'concavity error',
      'concave points error', 'symmetry error',
      'fractal dimension error', 'worst radius', 'worst texture',
      'worst perimeter', 'worst area', 'worst smoothness',
      'worst compactness', 'worst concavity', 'worst concave points',
      'worst symmetry', 'worst fractal dimension'], dtype='<U23')
```

In [12]:

```
features_labels = np.append(features, 'label')
```

In [13]:

```
breast_df.columns = features_labels
```

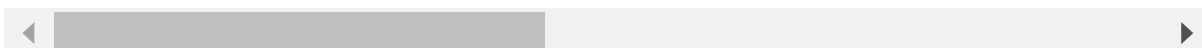
In [14]:

```
#make sure the columns are titled correctly
breast_df.head()
```

Out[14]:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809

5 rows × 31 columns



In [15]:

```
#replace the labels so its easier to read
breast_df['label'].replace(0, 'Benign', inplace=True)
breast_df['label'].replace(1, 'Malignant', inplace=True)
breast_df.tail()
```

Out[15]:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry
564	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24390	0.13890	0.17
565	20.13	28.25	131.20	1261.0	0.09780	0.10340	0.14400	0.09791	0.17
566	16.60	28.08	108.30	858.1	0.08455	0.10230	0.09251	0.05302	0.15
567	20.60	29.33	140.10	1265.0	0.11780	0.27700	0.35140	0.15200	0.20
568	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00000	0.00000	0.15

5 rows × 31 columns

In [16]:

```
x = breast_df.loc[:, features].values

#normalising the data
x = StandardScaler().fit_transform(x)
```

In [17]:

x.shape

Out[17]:

(569, 30)

In [18]:

```
#check the data is fit to be analysed
np.mean(x), np.std(x)
```

Out[18]:

(-6.826538293184326e-17, 1.0)

In [19]:

```
#convert the normalised data into tabular
feat_col = ['feature'+str(i) for i in range(x.shape[1])]
```

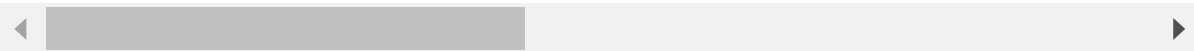
In [20]:

```
normalised_breast = pd.DataFrame(x, columns=feat_col)
normalised_breast.tail()
```

Out[20]:

	feature0	feature1	feature2	feature3	feature4	feature5	feature6	feature7	feat
564	2.110995	0.721473	2.060786	2.343856	1.041842	0.219060	1.947285	2.320965	-0.31
565	1.704854	2.085134	1.615931	1.723842	0.102458	-0.017833	0.693043	1.263669	-0.21
566	0.702284	2.045574	0.672676	0.577953	-0.840484	-0.038680	0.046588	0.105777	-0.80
567	1.838341	2.336457	1.982524	1.735218	1.525767	3.272144	3.296944	2.658866	2.13
568	-1.808401	1.221792	-1.814389	-1.347789	-3.112085	-1.150752	-1.114873	-1.261820	-0.82

5 rows × 30 columns



In [21]:

```
#begin the pca analysis
pca_breast = PCA(n_components=2)
principalComponents_breast = pca_breast.fit_transform(x)
```

In [22]:

```
#create a dataframe
principal_breast_df = pd.DataFrame(data = principalComponents_breast
                                   , columns = ['principal component 1', 'principal component 2'])
```

In [23]:

```
principal_breast_df.head()
```

Out[23]:

	principal component 1	principal component 2
0	9.192837	1.948583
1	2.387802	-3.768172
2	5.733896	-1.075174
3	7.122953	10.275589
4	3.935302	-1.948072

In [24]:

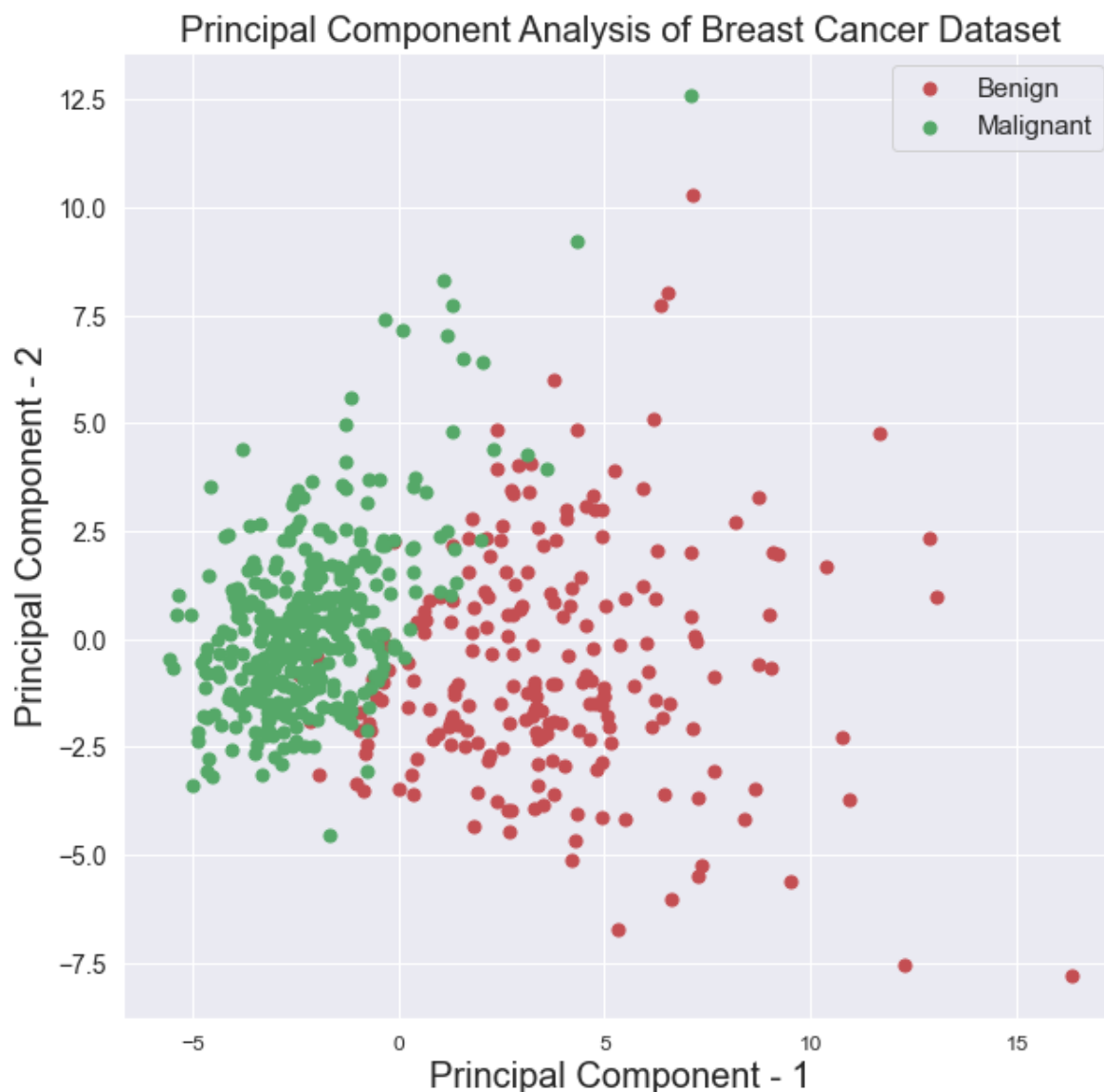
```
#visualise the data
plt.figure()
plt.figure(figsize=(10,10))
plt.xticks(fontsize=12)
plt.yticks(fontsize=14)
plt.xlabel('Principal Component - 1',fontsize=20)
plt.ylabel('Principal Component - 2',fontsize=20)
plt.title("Principal Component Analysis of Breast Cancer Dataset",fontsize=20)
targets = ['Benign', 'Malignant']
colors = ['r', 'g']
for target, color in zip(targets,colors):
    indicesToKeep = breast_df['label'] == target
    plt.scatter(principal_breast_df.loc[indicesToKeep, 'principal component 1']
                , principal_breast_df.loc[indicesToKeep, 'principal component 2'], c = color)

plt.legend(targets,prop={'size': 15})
```

Out[24]:

<matplotlib.legend.Legend at 0x1dc6ad56f70>

<Figure size 432x288 with 0 Axes>



In [25]:

```
breast_df.to_csv("breast_cancer_data.csv")
```

In [26]:

```
pca_breast = PCA(n_components=3)
principalComponents_breast = pca_breast.fit_transform(x)
```

In [27]:

```
print(principalComponents_breast)
```

```
[[ 9.1928368  1.94858256 -1.12316527]
 [ 2.38780177 -3.76817231 -0.5292925 ]
 [ 5.73389628 -1.07517377 -0.55174775]
 ...
 [ 1.25617929 -1.90229647  0.56273023]
 [10.37479407  1.67201034 -1.87702963]
 [-5.47524333 -0.67063743  1.49044381]]
```

In [28]:

```
import numpy
import sklearn

pdf = pd.DataFrame(data = principalComponents_breast,
                   columns=['PCA1', 'PCA2', 'PCA3'])

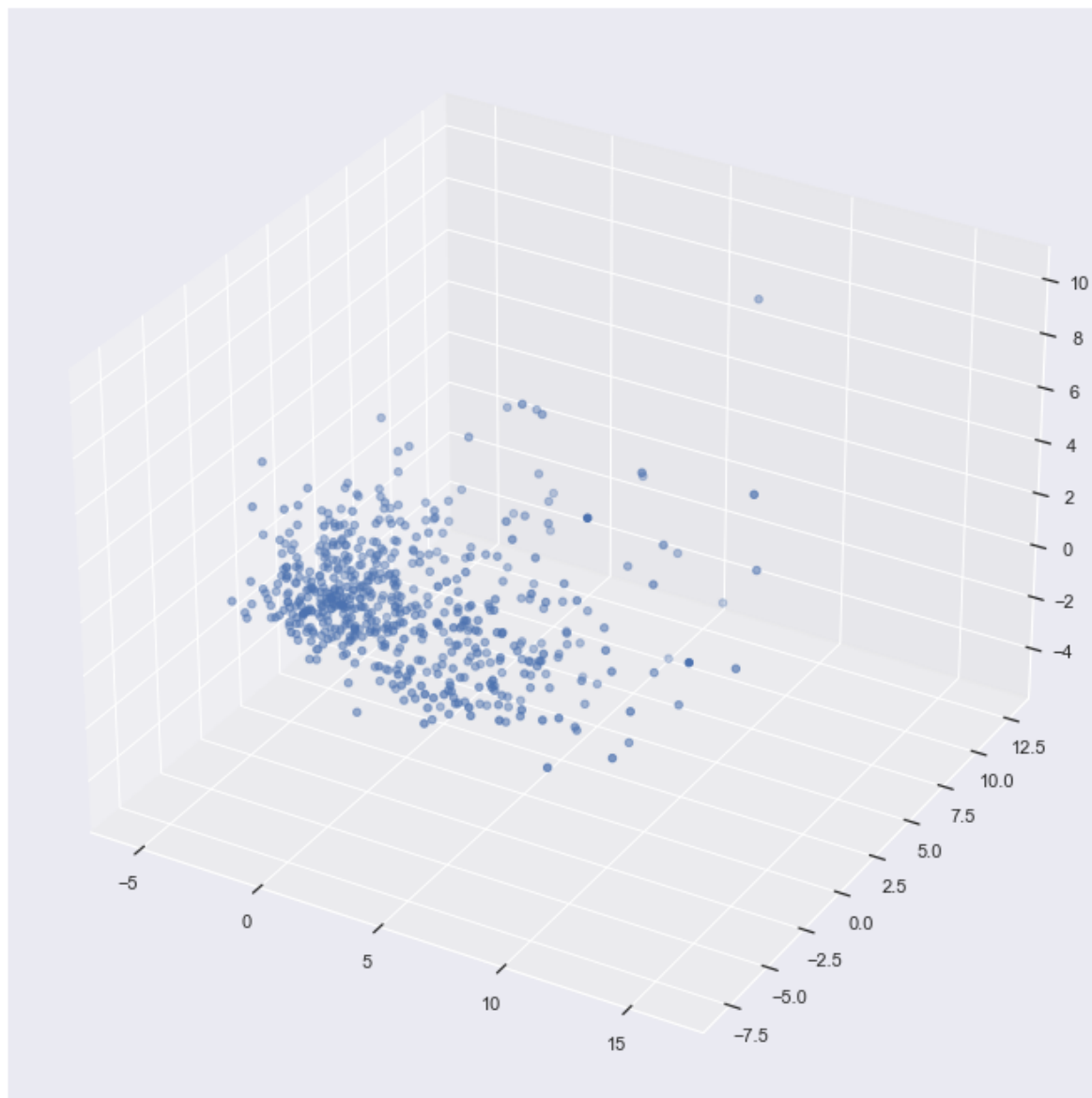
pdf.to_csv('df.csv')
```

In [29]:

```
fig = plt.figure(figsize=(12, 12))  
ax = fig.add_subplot(projection='3d')  
  
ax.scatter(pdf['PCA1'],pdf['PCA2'],pdf['PCA3'])
```

Out[29]:

<mpl_toolkits.mplot3d.art3d.Path3DCollection at 0x1dc6ade3910>



In [30]:

pca_breast.components_

Out[30]:

```
array([[ 0.21890244,  0.10372458,  0.22753729,  0.22099499,  0.14258969,
         0.23928535,  0.25840048,  0.26085376,  0.13816696,  0.06436335,
         0.20597878,  0.01742803,  0.21132592,  0.20286964,  0.01453145,
         0.17039345,  0.15358979,  0.1834174 ,  0.04249842,  0.10256832,
         0.22799663,  0.10446933,  0.23663968,  0.22487053,  0.12795256,
         0.21009588,  0.22876753,  0.25088597,  0.12290456,  0.13178394],
        [-0.23385713, -0.05970609, -0.21518136, -0.23107671,  0.18611303,
         0.15189161,  0.06016536, -0.0347675 ,  0.19034877,  0.36657547,
        -0.10555215,  0.08997968, -0.08945724, -0.15229263,  0.20443045,
         0.2327159 ,  0.19720729,  0.13032155,  0.183848 ,  0.28009203,
        -0.21986638, -0.0454673 , -0.19987843, -0.21935186,  0.17230435,
         0.14359318,  0.09796411, -0.00825724,  0.14188335,  0.27533946],
        [-0.00853125,  0.0645499 , -0.00931422,  0.02869952, -0.10429192,
        -0.07409157,  0.00273384, -0.02556354, -0.04023994, -0.02257407,
         0.26848139,  0.37463367,  0.26664537,  0.21600653,  0.30883898,
         0.15477971,  0.17646374,  0.22465757,  0.28858429,  0.21150376,
        -0.04750699, -0.04229782, -0.04854651, -0.01190231, -0.25979761,
        -0.23607563, -0.17305734, -0.17034407, -0.27131264, -0.23279131]])
```

In [31]:

```
from sklearn import datasets, neighbors, linear_model

X_digits, y_digits = datasets.load_digits(return_X_y=True)
X_digits = X_digits / X_digits.max()

n_samples = len(X_digits)

X_train = X_digits[: int(0.9 * n_samples)]
y_train = y_digits[: int(0.9 * n_samples)]
X_test = X_digits[int(0.9 * n_samples) :]
y_test = y_digits[int(0.9 * n_samples) :]

knn = neighbors.KNeighborsClassifier()
logistic = linear_model.LogisticRegression(max_iter=1000)

print("KNN score: %f" % knn.fit(X_train, y_train).score(X_test, y_test))
print(
    "LogisticRegression score: %f"
    % logistic.fit(X_train, y_train).score(X_test, y_test))
```

KNN score: 0.961111

LogisticRegression score: 0.933333

